Fundamentos de Lenguajes de Programación

Cap. 1 Introducción a la Computadora y Programación

Texto: Starting Out with Python, Gaddis

Outline

- Definir conceptos fundamentales
  - Programa
  - Lenguaje de Programación
  - Oración (*statement*)
- Tipos de Oraciones
- Estructuras de Flujo Control

Conceptos Fundamentales

- Un programa de computadora es una colección de oraciones que son ejecutadas por una computadora para llevar a cabo una tarea específica
  - Un programa está escrito en un lenguaje de programación

Conceptos Fundamentales

- Un programa de computadora es una colección de oraciones que son ejecutadas por una computadora para llevar a cabo una tarea específica
  - Un programa está escrito en un lenguaje de programación

- Un lenguaje de programación es un conjunto de palabras reservadas y otros componentes y reglas gramaticales que rigen la codificación de oraciones correctas

Conceptos Fundamentales

- Un programa de computadora es una colección de oraciones que son ejecutadas por una computadora para llevar a cabo una tarea específica
  - Un programa está escrito en un lenguaje de programación

- Un lenguaje de programación es un conjunto de palabras reservadas y otros componentes y reglas gramaticales que rigen la codificación de oraciones correctas
  - Usualmente se refiere a lenguajes de alto nivel
  - Tiene dos dimensiones:
    - Sintaxis
    - Semántica

Conceptos Fundamentales

- La sintaxis de un lenguaje de programación es el conjunto de reglas para combinar los diferentes componentes del lenguaje de tal forma que generen oraciones sintácticamente correctas

Conceptos Fundamentales

- La sintaxis de un lenguaje de programación es el conjunto de reglas para combinar los diferentes componentes del lenguaje de tal forma que generen oraciones sintácticamente correctas

- La semántica de un lenguaje de programación es el significado de las oraciones sintácticamente correctas
  - Describe el proceso que sigue la computadora al ejecutar las oraciones que componen un programa

Conceptos Fundamentales

- Una oración o *statement* de un lenguaje de programación es una unidad sintáctica que expresa alguna acción que debe ser llevado a cabo por la computadora

Conceptos Fundamentales

- Una oración o *statement* de un lenguaje de programación es una unidad sintáctica que expresa alguna acción que debe ser llevado a cabo por la computadora

- Estas están compuestas por
  - Palabras reservadas
  - Operadores y Operandos
  - Funciones
  - Símbolos de puntuación

Tipos de Oraciones

1. Oraciones de Entrada (*Input*)
2. Oraciones de Salida (*Output*)
3. Oraciones de Asignación (*Assignment*)
4. Oraciones de Control
   - Selección o Decisión
   - Repetición
   - Subprograma

Oraciones de Entrada

- La oración en la línea 1 está compuesta de
  - Variable de tipo string (`nombre`)
  - Operador de asignación (=)
  - Función *built-in* (`input`)
  - Argumento de la función `input` (“`ingrese su nombre`”)
    - Este es un literal de tipo string

```
1. nombre = input("ingrese su nombre")
2.
3.
4. no_se_que = input()
5.
```

Oraciones de Salida

- La oración en la línea 1 está compuesta por
  - Función *built-in* (`print`)
  - Argumento de la función `print` (“`Hello world`”)
    - Este es un literal de tipo string

```
1. print("Hello world")
2.
```

Oraciones de Salida

- La oración en la línea 1 está compuesta por
  - Función *built-in* (`print`)
  - Argumento de la función `print` (“`Hello world`”)
    - Este es un literal de tipo string

```
1. print("Hello world")
2.
```

- La función `print` hace que sus argumentos se desplieguen en la pantalla

```
Hello world
```

Oraciones de Asignación

- El formato para una oración e asignación es:
  - variable = expresión

```
1. edad = 21
2. nombre = "Juan del Pueblo"
3.
4. ancho = 3.0
5. largo = 1.5
6. perimetro = 2.0 * (ancho +
   largo)
7.
8. a = b = c = 0
9. x, y = 1, 2
10. x, y = y, x
11.
12. apodo = input("ingrese su
   apodo")
13.
```

Oraciones de Asignación

- El formato para una oración e asignación es:
  - variable = expresión
- Note que las variables son de diferentes tipos de datos
  - `edad` es de tipo `int`
  - `nombre` y `apodo` son de tipo `string`
  - `ancho`, `largo` y `perimetro` son de tipo `float`

```
edad = 21
nombre = "Juan del Pueblo"

ancho = 3.0
largo = 1.5
perimetro = 2.0 * (ancho + largo)

a = b = c = 0
x, y = 1, 2
x, y = y, x

apodo = input("ingrese su apodo")

```

Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.

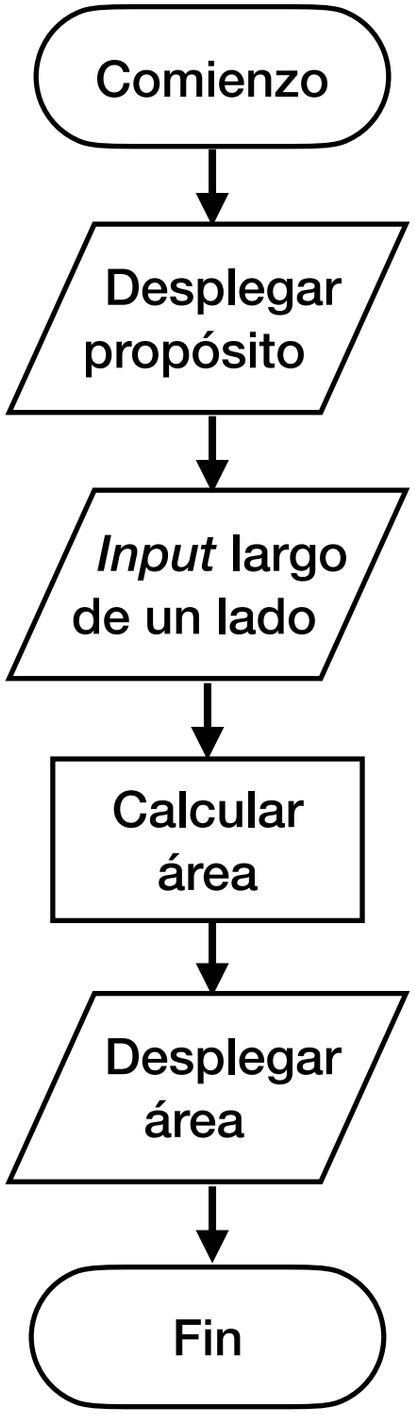
Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial

```
1. print("Calculemos el area de un cuadrado!")
2. largo = int(input("Entre el largo de un lado: "))
3. area = largo * largo
4. print("El area del cuadrado es", area)
5.
```

Estructuras de Flujo Control

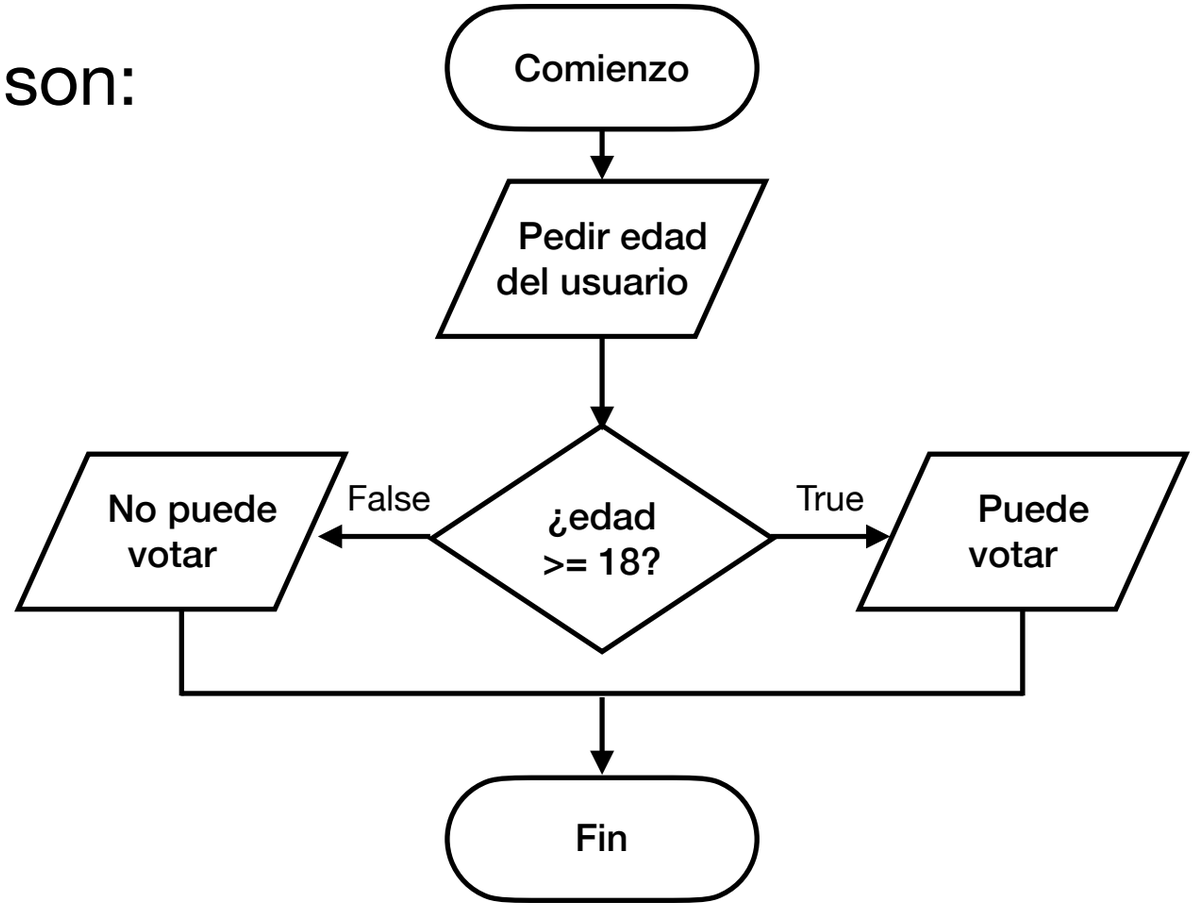
- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial




```
1. print("Calculemos el area de un cuadrado!")
2. largo = int(input("Entre el largo de un lado: "))
3. area = largo * largo
4. print("El area del cuadrado es", area)
5.
```

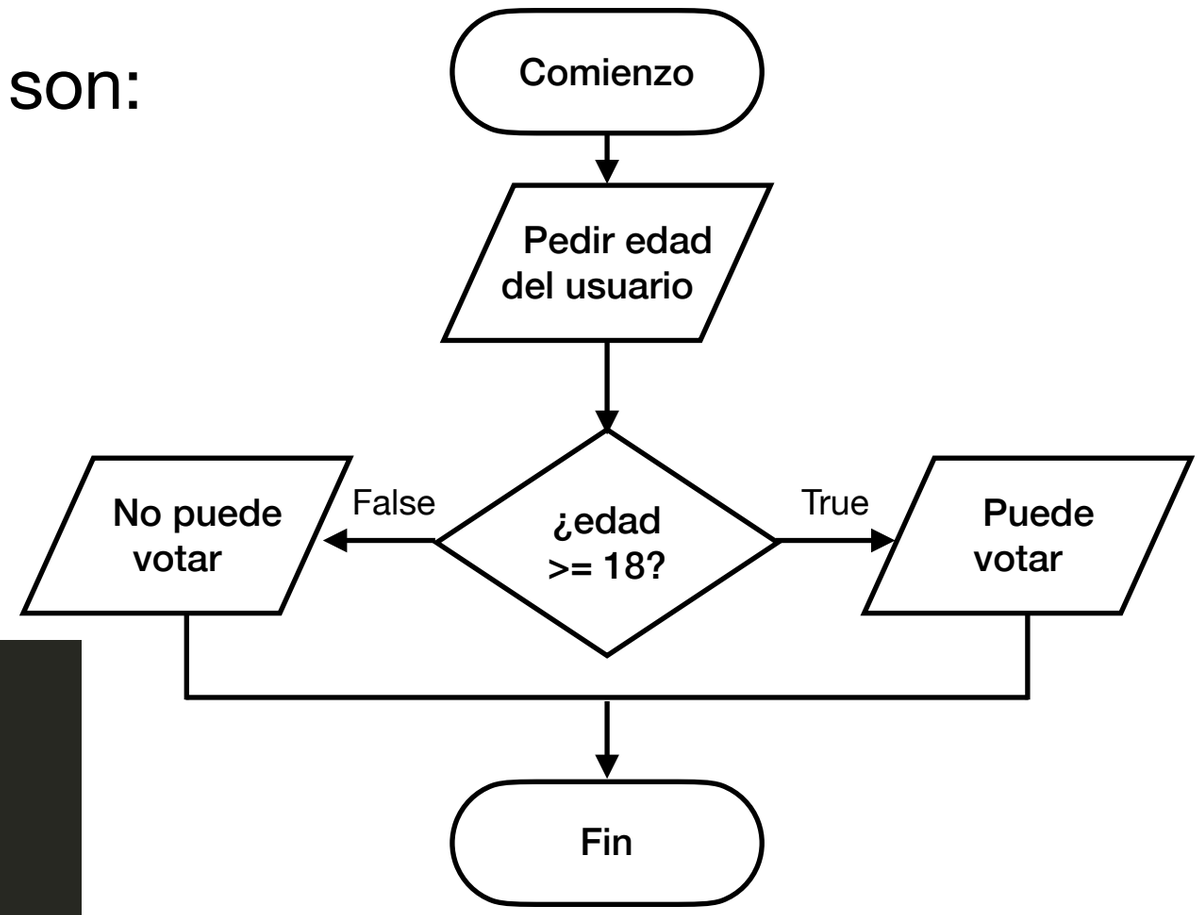
Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión



Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión

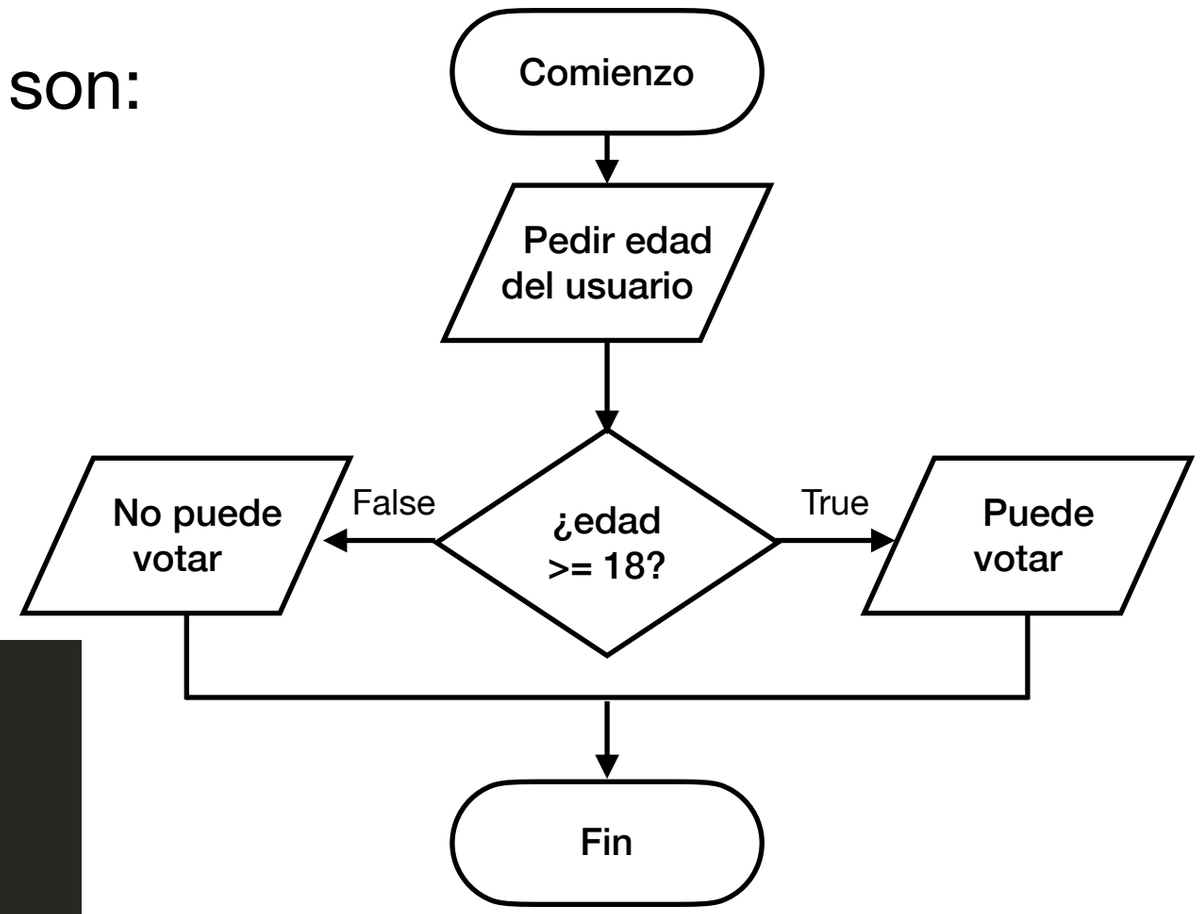



```python
edad = input("Entre su edad: ")
edad = int(edad)
if edad >= 18:
    print("Puede votar!")
else:
    print("No puede votar.")

```

Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión




```
edad = input("Entre su edad: ")
edad = int(edad)
if edad >= 18:
    print("Puede votar!")
else:
    print("No puede votar.")

```

Las palabras `if` y `else` son reservadas

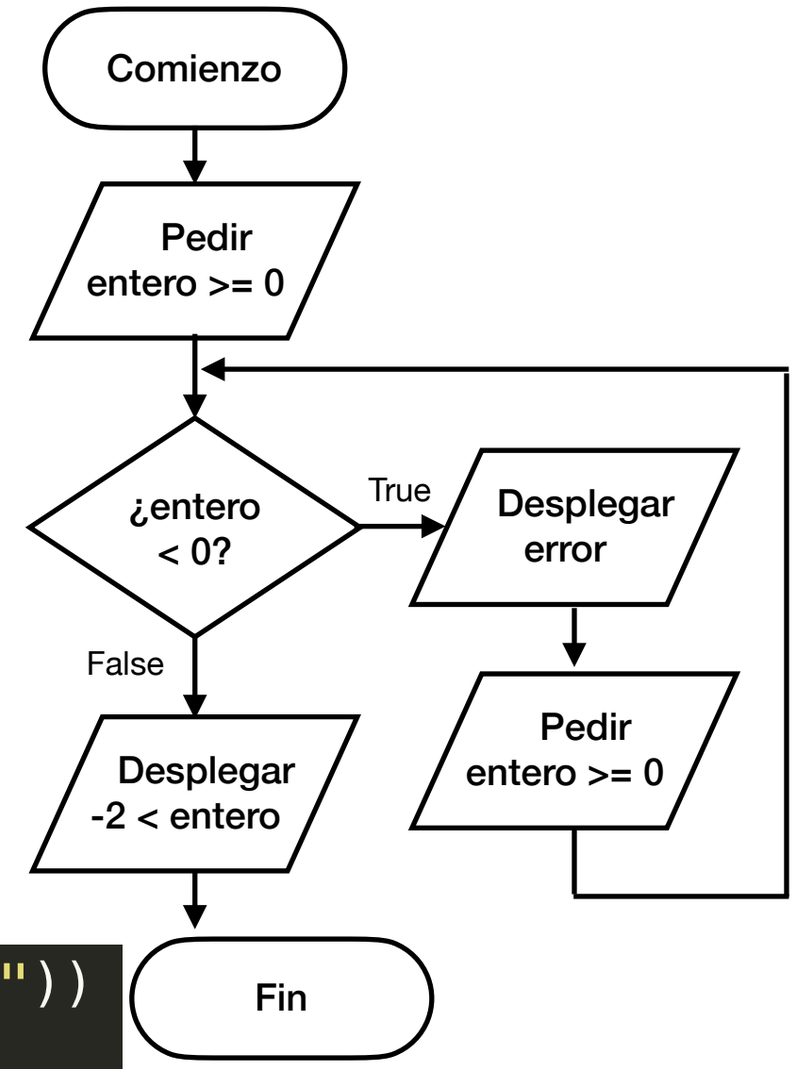
Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión
    3. Repetición

```
1. entero = int(input("Entre un número entero >= 0: "))
2. while entero < 0:
3.     print("El número debe ser >= 0.")
4.     entero = int(input("Entre otro número: “))
5. print("El número -2 es menor que", entero)
6.
```

Estructuras de Flujo Control

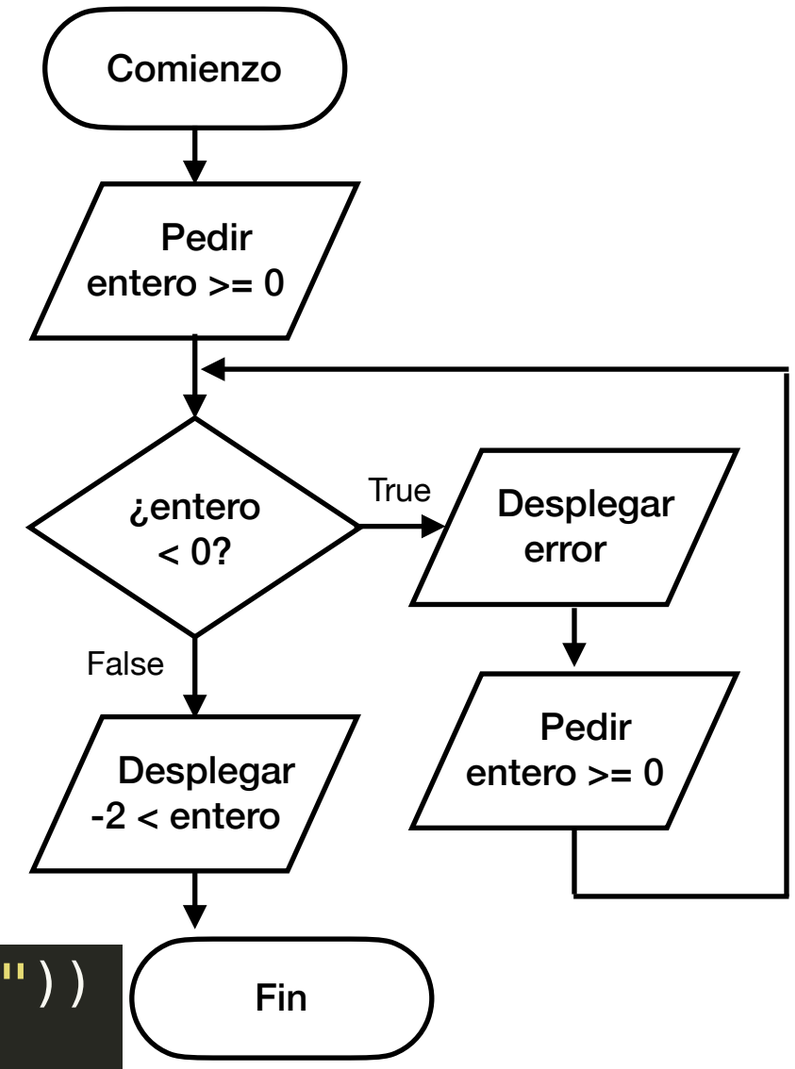
- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión
    3. Repetición




```
1. entero = int(input("Entre un número entero >= 0: "))
2. while entero < 0:
3.     print("El número debe ser >= 0.")
4.     entero = int(input("Entre otro número: “))
5. print("El número -2 es menor que", entero)
6.
```

Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión
    3. Repetición



La palabra `while` es reservada

```
1. entero = int(input("Entre un número entero >= 0: "))
2. while entero < 0:
3.     print("El número debe ser >= 0.")
4.     entero = int(input("Entre otro número: “))
5. print("El número -2 es menor que", entero)
6.
```

Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión
    3. Repetición
    4. Subprogramas

```
1. def main():
2.     mensaje = input("Escriba algo: ")
3.     desplegar(mensaje)
4.
5. def desplegar(texto):
6.     print(texto)
7.
8. main()
9.
```

Estructuras de Flujo Control

- Una estructura de flujo control es un diseño lógico que controla el orden en que un conjunto de oraciones se ejecutarán.
- Las cuatro estructuras de flujo control son:
    1. Secuencial
    2. Selección o Decisión
    3. Repetición
    4. Subprogramas

```
1. def main():
2.     mensaje = input("Escriba algo: ")
3.     desplegar(mensaje)
4.
5. def desplegar(texto):
6.     print(texto)
7.
8. main()
9.
```

La palabra *def* es reservada

Tipos de Oraciones

1. Oraciones de Entrada (*Input*)
2. Oraciones de Salida (*Output*)
3. Oraciones de Asignación (*Assignment*)
4. Oraciones de Control
   - Selección o Decisión
   - Repetición
   - Subprograma

Componentes de Oraciones

- Palabras reservadas

Keywords

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Componentes de Oraciones

- Palabras reservadas

Keywords

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

- Operadores
  - Asignación: =
  - Aritméticos: +, −, ∗, /, //, %, ∗∗
  - Relacionales: >, >=, <, <=, ==, !=
  - Lógicos: `and`, `or`, `not`

Componentes de Oraciones

- Palabras reservadas
- Operadores
  - Asignación: =
  - Aritméticos: +, −, ∗, /, //, %, ∗∗
  - Relacionales: >, >=, <, <=, ==, !=
  - Lógicos: `and`, `or`, `not`
- Operandos
  - Literales, variables y constantes

Keywords

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Componentes de Oraciones

- Palabras reservadas
- Operadores
  - Asignación: =
  - Aritméticos: +, −, ∗, /, //, %, ∗∗
  - Relacionales: >, >=, <, <=, ==, !=
  - Lógicos: `and`, `or`, `not`
- Operandos
  - Literales, variables y constantes
- Funciones
  - `print`, `input`, `int`,...

Keywords

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Componentes de Oraciones

- Palabras reservadas
- Operadores
  - Asignación: =
  - Aritméticos: +, −, ∗, /, //, %, ∗∗
  - Relacionales: >, >=, <, <=, ==, !=
  - Lógicos: `and`, `or`, `not`
- Operandos
  - Literales, variables y constantes
- Funciones
  - `print`, `input`, `int`,...
- Símbolos de puntuación: `. : ( ) [ ] { }`

Keywords

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Componentes de Oraciones

Keywords

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

- Palabras reservadas
- Operadores
  - Asignación: =
  - Aritméticos: +, −, ∗, /, //, %, ∗∗
  - Relacionales: >, >=, <, <=, ==, !=
  - Lógicos: `and`, `or`, `not`
- Operandos
  - Literales, variables y constantes
- Funciones
  - `print`, `input`, `int`,...
- Símbolos de puntuación: `. : ( ) [ ] { }`

Repaso

- ¿Qué es un programa?

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación
- ¿Cuáles son los componentes de una oración?

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación
- ¿Cuáles son los componentes de una oración?
- ¿Qué es una estructura de flujo control?

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación
- ¿Cuáles son los componentes de una oración?
- ¿Qué es una estructura de flujo control?
- ¿Cuáles son las cuatro estructuras de flujo control?

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación
- ¿Cuáles son los componentes de una oración?
- ¿Qué es una estructura de flujo control?
- ¿Cuáles son las cuatro estructuras de flujo control?
- ¿Cuáles son los diferentes tipos de oraciones?

Repaso

- ¿Qué es un programa?
- Defina una oración de un lenguaje de programación
- ¿Cuáles son los componentes de una oración?
- ¿Qué es una estructura de flujo control?
- ¿Cuáles son las cuatro estructuras de flujo control?
- ¿Cuáles son los diferentes tipos de oraciones?
- Explique qué es el ciclo *Fetch-Decode-Execute*

Repaso

- ¿Cuál es el valor de la variable b luego de ejecutar el siguiente código?

```
1. a = 10
2. b = a
3. a = 20
4.
```

Repaso

- ¿Cuál es el valor de la variable b luego de ejecutar el siguiente código?
- ¿Cuál(es) estructura(s) de flujo control se utilizan en el código?

```
1. a = 10
2. b = a
3. a = 20
4.
```

Repaso

- ¿Cuál es el valor de la variable b luego de ejecutar el siguiente código?
- ¿Cuál(es) estructura(s) de flujo control se utilizan en el código?
- ¿Qué tipos de oraciones tiene el código?

```
1. a = 10
2. b = a
3. a = 20
4.
```

Repaso

- ¿Cuál es el valor de la variable b luego de ejecutar el siguiente código?
- ¿Cuál(es) estructura(s) de flujo control se utilizan en el código?
- ¿Qué tipos de oraciones tiene el código?
- ¿De qué tipo de dato es la variable a?

```
1. a = 10
2. b = a
3. a = 20
4.
```

Repaso

- ¿Cuál es el valor de la variable b luego de ejecutar el siguiente código?
- ¿Cuál(es) estructura(s) de flujo control se utilizan en el código?
- ¿Qué tipos de oraciones tiene el código?
- ¿De qué tipo de dato es la variable a?
- ¿De qué tipo de dato es la variable b?

```
1. a = 10
2. b = a
3. a = 20
4.
```